UNITY 6000.0.62F1 · COMPLETE OFFLINE DOCUMENTATION

Upside Solitaire Developer Guide

Install. Edit. Reskin. Build your release.

szekipapa77@gmail.com



CHAPTER 01

Start here: Upside Solitaire

Upside Solitaire is a standalone editable Unity game template. Play backwards Klondike in a clockmaker workshop. Wind the four foundation dials from King down to Ace. Build tableau columns upward in alternating colours; only Aces start empty columns. Draw stock cards and follow the current deal goal and limits.

Includes 25 curated levels, source-backed generation, a game-specific visual editor and validation, 4 guided helpers, local saving and portrait/landscape scenes. Keep the required private game Kit and publisher-owned Szekipapa77Ads layer. Provider SDK binaries are excluded. There is no bundled HLK, purchases, multiplayer server or remote LiveOps backend. Import only one game Kit into a project because the Kit classes share names.

CHAPTER 02

Requirements, installation and first play

Use Unity **6000.0.62f1**, URP **17.0.4** and Unity UI **2.0.0**. Install matching Android SDK/NDK/OpenJDK and WebGL modules in Unity Hub. Retain the supplied manifest and ProjectSettings.

1. Extract the standalone project to a short writable local path, then open it in the matching editor. Wait for package import and compilation.
2. For native unitypackage import, create a matching URP project and use Assets > Import Package > Custom Package. Import every game, Kit, Szekipapa77Ads and WebGL-template entry.
3. Run Szekipapa77 > Upside Solitaire > Setup. It configures rendering and player settings; review its changes when integrating with existing work.
4. Active Input Handling must support the classic Input Manager: choose Old or Both. New Input System only is insufficient.
5. Open `Assets/Games/ReverseSol/Scenes/ReverseSol_Mobile.unity` and press Play. Follow the level card and first-level instructions.
6. Stop Play mode before editing level assets. Keep all .meta GUIDs. The game uses classic Unity UI Text; do not assume an extra TextMesh Pro import is required.

CHAPTER 03

Project folders and exact source map

All game-relative paths below start at `Assets/Games/ReverseSol`. Runtime UI is built in code; editing an unrelated static Canvas does not reskin it. Scripts contains game rules and views; Editor contains authoring tools; Kit supplies UI/audio/save/platform support; Resources/reversesol contains level JSON, art, music and sound; Settings owns URP assets; Scenes contains both entry scenes.

- `Scripts/ReverseSolGame.cs`
- `Scripts/ReverseSolGame.Debug.cs`
- `Scripts/ReverseSolGame.Hud.cs`
- `Scripts/ReverseSolGame.Layout.cs`
- `Scripts/RvBoosters.cs`
- `Scripts/RvCardView.cs`
- `Scripts/RvCore.cs`
- `Scripts/RvGen.cs`
- `Scripts/RvLevels.cs`
- `Scripts/RvSolver.cs`
- `Scripts/RvTheme.cs`

Main flow: ReverseSolGame.cs. UI and popup consumers: ReverseSolGame.Hud.cs. Model/generator/loading: RvBoosters.cs, RvCore.cs, RvGen.cs, RvLevels.cs. Authoring: Editor/ReverseSolLevelEditor.cs. Shared ad settings live outside the game at Assets/Szekipapa77Ads/Resources/AdsSettings.asset.

CHAPTER 04

Gameplay, controls and progression

Play backwards Klondike in a clockmaker workshop. Wind the four foundation dials from King down to Ace. Build tableau columns upward in alternating colours; only Aces start empty columns. Draw stock cards and follow the current deal goal and limits.

The initial bank has 25 curated levels. Read the supplied loader/generator before changing continuation rules. Play the guided opening, ordinary progress, completion and retry paths; preserve saved indexes during a visual reskin. The Mobile scene is portrait and the WebGL-named scene is landscape, but the supplied Build menu selects portrait for both build targets. Select landscape explicitly in your Build Profile if desired. Test a near-startup scene replacement as well as a normal restart.

CHAPTER 05

Tutorials and every helper

Fresh local progress triggers the first-level guidance. Newly unlocked helpers have introduction and guided-use flows. Reset only a test save when reviewing tutorials. A free guided first use can differ from ordinary charge consumption. Exact helper names/unlocks/charges: Scripts/RvBoosters.cs; help and effect descriptions: Scripts/RvBoosters.cs.

Helper	Unlock	Free charges	Action and effect
Rewind	2	3	Winds back your last move.
Loupe	4	3	Shows a good next move.
Tweezers	7	2	Pull any face-down card out onto the waste.
Pocket	11	2	Opens a spare slot to park one card for the rest of the level.

Activate the tool and perform its required target action: arming a tool alone does not demonstrate its effect. A successful use requires the stated preconditions from this game: show both activation and its actual effect. Exhausted charges may offer a rewarded ad; verify success/cancel/failure and grant a reward once only.

CHAPTER 06

Visual level editor: worked editing session

Open **Szekipapa77 > Upside Solitaire > Level Editor**. Exact implementation: `Editor/ReverseSolLevelEditor.cs`.

Duplicate an early entry in this game-specific visual editor, change its visible name and one rule, save, validate, then play it. Use the exact source-control table below for this game.

1. Back up Resources/reversesol/Levels/levels.json.
2. Duplicate a simple early level and edit one small element.
3. Use the game-specific validation controls and resolve every reported issue.
4. Save explicitly; stop and restart Play mode, then use Play this level or navigate to the edited index.
5. Check portrait and landscape and rebuild demo outputs after accepting data changes.

The exact current authoring controls, including tooltips and generator switches, follow. These are game-specific source references, not a generic editor feature list.

Source	Exact current code
`ReverseSolLevelEditor.cs:41`	`[MenuItem("Szekipapa77/Upside Solitaire/Level Editor")]`
`ReverseSolLevelEditor.cs:106`	`if (GUILayout.Button(_dirty ? "Save *" : "Save", EditorStyles.toolbarButton, GUILayout.Width(70))) Save();`
`ReverseSolLevelEditor.cs:107`	`if (GUILayout.Button("Reload", EditorStyles.toolbarButton, GUILayout.Width(60)) && (!_dirty || EditorUtility.DisplayDialog("Reload", "Discard unsaved changes?", "Discard", "Cancel"))) Load();`
`ReverseSolLevelEditor.cs:109`	`if (GUILayout.Button("Validate level", EditorStyles.toolbarButton, GUILayout.Width(100))) ValidateOne(_data.selected);`
`ReverseSolLevelEditor.cs:110`	`if (GUILayout.Button("Validate all", EditorStyles.toolbarButton, GUILayout.Width(85))) ValidateAll();`
`ReverseSolLevelEditor.cs:113`	`if (GUILayout.Button("Play this level", EditorStyles.toolbarButton, GUILayout.Width(105))) PlayLevel();`

ReverseSolLevelEditor.cs:134	`if (GUILayout.Button("<b>" + (i + 1) + ".</b> " + d.name + " <color=#9ab>(" + d.goal + ")</color>" + tags + diff + res, style, GUILayout.Height(22)))`
ReverseSolLevelEditor.cs:139	`if (GUILayout.Button("Add")) { Rec("Add level"); int at = Mathf.Min(_data.selected + 1, File.levels.Count); File.levels.Insert(at, new RvLevelDef { name = "New clock", seed = Environment.TickCount & 0xffff }); _data.selected = at; _results.Clear(); }`
ReverseSolLevelEditor.cs:141	`if (GUILayout.Button("Dup")) { Rec("Duplicate level"); var c = Cur.Clone(); c.name += " copy"; File.levels.Insert(_data.selected + 1, c); _data.selected++; _results.Clear(); }`
ReverseSolLevelEditor.cs:142	`if (GUILayout.Button("Del") && EditorUtility.DisplayDialog("Delete level", "Delete level " + (_data.selected + 1) + " (" + Cur.name + ")?", "Delete", "Cancel")) { Rec("Delete level"); File.levels.RemoveAt(_data.selected); _data.selected = Mathf.Clamp(_data.selected, 0, File.levels.Count - 1); _results.Clear(); }`
ReverseSolLevelEditor.cs:143	`if (GUILayout.Button("Up") && _data.selected > 0) { Rec("Move level"); int s = _data.selected; (File.levels[s], File.levels[s - 1]) = (File.levels[s - 1], File.levels[s]); _data.selected--; _results.Clear(); }`
ReverseSolLevelEditor.cs:144	`if (GUILayout.Button("Down") && _data.selected < File.levels.Count - 1) { Rec("Move level"); int s = _data.selected; (File.levels[s], File.levels[s + 1]) = (File.levels[s + 1], File.levels[s]); _data.selected++; _results.Clear(); }`
ReverseSolLevelEditor.cs:148	`if (_results.TryGetValue(_data.selected, out var line)) EditorGUILayout.HelpBox(line, line.StartsWith("ERR", StringComparison.Ordinal) ? MessageType.Error : MessageType.Info);`
ReverseSolLevelEditor.cs:271	`if (GUILayout.Button("Clear all pins")) { Rec("Clear pins"); Cur.pins.Clear(); }`
ReverseSolLevelEditor.cs:272	`if (GUILayout.Button("Re-roll seed")) { Rec("Re-roll seed"); Cur.seed = UnityEngine.Random.Range(1, 999999); }`

ReverseSolLevelEditor.cs:273	`if (GUILayout.Button("Kings to the first stock cards")) KingsUpFront();`
ReverseSolLevelEditor.cs:324	`string name = EditorGUILayout.TextField("Name", d.name);`
ReverseSolLevelEditor.cs:325	`EditorGUILayout.LabelField("First-time tip");`
ReverseSolLevelEditor.cs:326	`string tip = EditorGUILayout.TextArea(d.tip ?? "", GUILayout.Height(48));`
ReverseSolLevelEditor.cs:327	`int seed = EditorGUILayout.IntField("Seed", d.seed);`
ReverseSolLevelEditor.cs:329	`gi = EditorGUILayout.Popup("Goal", gi, GoalNames);`
ReverseSolLevelEditor.cs:331	`if (gi == 1) column = EditorGUILayout.IntSlider("Column to clear", d.column, 2, 7);`
ReverseSolLevelEditor.cs:334	`dialSuit = EditorGUILayout.Popup("Dial suit", d.dialSuit, SuitNames);`
ReverseSolLevelEditor.cs:335	`dialRank = EditorGUILayout.IntSlider(new GUIContent("Down to rank", "1 = Ace (whole suit), 12 = Queen"), d.dialRank, 1, 12);`
ReverseSolLevelEditor.cs:337	`int moves = EditorGUILayout.IntField(new GUIContent("Move limit", "0 = none"), d.moves);`
ReverseSolLevelEditor.cs:338	`int time = EditorGUILayout.IntField(new GUIContent("Time limit (s)", "0 = none"), d.time);`
ReverseSolLevelEditor.cs:339	`bool open = EditorGUILayout.Toggle(new GUIContent("Open Face deal", "Every tableau card face up; only free runs move"), d.open);`
ReverseSolLevelEditor.cs:340	`bool breather = EditorGUILayout.Toggle("Breather", d.breather);`
ReverseSolLevelEditor.cs:347	`EditorGUILayout.LabelField("Bells", d.bells.Count == 0 ? "-" : string.Join(", ", d.bells));`
ReverseSolLevelEditor.cs:348	`EditorGUILayout.LabelField("Domes", d.domes.Count == 0 ? "-" : string.Join(", ", d.domes));`

ReverseSolLevelEditor.cs:349	EditorGUILayout.LabelField("Pins", d.pins.Count == 0 ? "-" : string.Join(", ", d.pins));
ReverseSolLevelEditor.cs:350	EditorGUILayout.LabelField("Solver", d.solverMoves > 0 ? d.solverMoves + " moves, " + d.nodes + " nodes, difficulty " + d.difficulty.ToString("0", CultureInfo.InvariantCulture) : "not validated");
ReverseSolLevelEditor.cs:355	if (GUILayout.Button("Validate")) ValidateOne(_data.selected);
ReverseSolLevelEditor.cs:356	if (GUILayout.Button(new GUIContent("Fit limits", "Solve without limits, then set the move/time budget from the solution (1.4x moves, 3s per move + 20s)"))) FitLimits();
ReverseSolLevelEditor.cs:419	_genCount = EditorGUILayout.IntSlider("Count", _genCount, 1, 30);
ReverseSolLevelEditor.cs:420	_genReplace = EditorGUILayout.Toggle("Replace range", _genReplace);
ReverseSolLevelEditor.cs:421	_genStart = EditorGUILayout.IntField(_genReplace ? "Start level (1-based)" : "Level number of first", _genStart);
ReverseSolLevelEditor.cs:422	_genSeed = EditorGUILayout.IntField("Seed", _genSeed);
ReverseSolLevelEditor.cs:423	_dStart = EditorGUILayout.Slider("Start difficulty", _dStart, 0, 1);
ReverseSolLevelEditor.cs:424	_dEnd = EditorGUILayout.Slider("End difficulty", _dEnd, 0, 1);
ReverseSolLevelEditor.cs:425	_dSaw = EditorGUILayout.Slider("Sawtooth climb", _dSaw, 0, 0.2f);
ReverseSolLevelEditor.cs:426	_genBreather = EditorGUILayout.IntSlider("Breather every", _genBreather, 2, 10);
ReverseSolLevelEditor.cs:427	_dDrop = EditorGUILayout.Slider("Breather drop", _dDrop, 0, 0.6f);

`ReverseSolLevelEditor.cs:428`	`_genCandidates = EditorGUILayout.IntSlider(new GUIContent("Candidates per level", "Solved deals tried per level; the one closest to the curve is kept"), _genCandidates, 1, 10);`
`ReverseSolLevelEditor.cs:438`	`if (GUILayout.Button(_genReplace ? "Generate and replace levels " + _genStart + "-" + (_genStart + _genCount - 1) : "Generate and append " + _genCount + " levels")) Generate();`

CHAPTER 07

Generator, insertion and replacement

Use this editor generation panel, choose a reproducible seed and small count, review start/index and replace behavior, then generate, save and reopen. The source excerpt below identifies exact append/replace semantics.

Source	Exact current code
`ReverseSolLevelEditor.cs:29`	`bool _genReplace;`
`ReverseSolLevelEditor.cs:420`	`_genReplace = EditorGUILayout.Toggle("Replace range", _genReplace);`
`ReverseSolLevelEditor.cs:421`	`_genStart = EditorGUILayout.IntField(_genReplace ? "Start level (1-based)" : "Level number of first", _genStart);`
`ReverseSolLevelEditor.cs:438`	`if (GUILayout.Button(_genReplace ? "Generate and replace levels " + _genStart + "-" + (_genStart + _genCount - 1) : "Generate and append " + _genCount + " levels")) Generate();`
`ReverseSolLevelEditor.cs:452`	`int index = (_genReplace ? _genStart : File.levels.Count + 1) + i;`
`ReverseSolLevelEditor.cs:460`	`if (_genReplace)`
`ReverseSolLevelEditor.cs:464`	`File.levels.RemoveRange(start, remove);`
`ReverseSolLevelEditor.cs:465`	`File.levels.InsertRange(start, made);`
`ReverseSolLevelEditor.cs:468`	`else { File.levels.AddRange(made); _data.selected = File.levels.Count - made.Count; }`

Worked extension: back up the bank, generate a small set at moderate difficulty using the exact per-game controls, validate them, then Save. Reopen the JSON and confirm the count changed. Do not assume every editor has the same insertion behavior: the actual implementation below distinguishes insert, append and replace. Runtime continuation does not automatically append new records to project JSON. Validation and bot scores help find problems but do not certify a pleasant human difficulty curve.

CHAPTER 08

Actual level JSON and identifiers

Canonical asset: `Assets/Games/ReverseSol/Resources/reversesol/Levels/levels.json`. Array order controls visible numbering. Preserve numeric/boolean types, root version fields where present and resource/save identifiers. The actual first record is shown below; use the editor for structured coordinates and block/word data.


```
 {
   "name": "Mainspring",
   "tip": "Wind the dials backwards: each dial takes its King first, then the Queen, down to the Ace. On the columns, build UP
in alternating colours. Tap a card to move it, or drag it.",
   "seed": 210458,
   "goal": "win",
   "column": 7,
   "dialSuit": 0,
   "dialRank": 7,
   "moves": 0,
   "time": 0,
   "open": false,
   "pins": [],
   "bells": [],
   "domes": [],
   "difficulty": 36.1,
   "breather": false,
   "solverMoves": 100,
   "nodes": 100,
   "par": 100
 }
```


Root keys: version, levels. Inspect RvBoosters.cs, RvCore.cs, RvGen.cs, RvLevels.cs for field semantics and validation. Keep visible names/hints distinct from technical keys. The game-specific board, wave, card, opponent or question records are gameplay data; UI translation is separate.

CHAPTER 09

Reskin recipe: interface colours

Stop Play mode and open `Scripts/RvTheme.cs`. Find `Ink`, currently `#2A2550`. Change that hex literal to `#42C9B0`, preserve its symbol/type, compile and reopen the scene. Review HUD, menus, helper cards, settings/results and both orientations. Art sprites and procedural fallback shapes can have additional colours; edit them deliberately.

Actual consumers of this symbol are listed first, then all current palette/hex assignments. Editor-only palettes do not control the runtime. Preserve state/goal contrast and rebuild after accepting a change.

Source	Exact current code
`RvTheme.cs:30 / public static Color SuitColor(int card) => RvCard.Red(card) ? Rose : Ink;`	`public static Color SuitColor(int card) => RvCard.Red(card) ? Rose : Ink;`
`RvTheme.cs:31 / public static Color SuitColorOf(int suit) => suit < 2 ? Rose : Ink;`	`public static Color SuitColorOf(int suit) => suit < 2 ? Rose : Ink;`

Source	Exact current code
`ReverseSolGame.Hud.cs:175 / void UpdateHud(bool full)`	`_movesText.color = movesLimit > 0 && movesLimit - S.Moves <= 10 ? HzUI.Hex("#FF8FA0") : Color.white;`
`ReverseSolGame.Hud.cs:189 / void UpdateClock()`	`_timeText.color = left <= 30 ? HzUI.Hex("#FF8FA0") : Color.white;`
`ReverseSolGame.Hud.cs:351 / void OpenSettings()`	`var cols = new[] { RvTheme.Plum, RvTheme.Plum, RvTheme.Copper, HzUI.Hex("#2E9C7E") };`
`ReverseSolGame.Layout.cs:149 / void UpdateDomes(bool animate)`	`_domeText[c].color = RvCard.Red(key) ? HzUI.Hex("#FF8FA0") : Color.white;`
`RvBoosters.cs:25 / class/static declarations`	`Color = HzUI.Hex("#C8693B"), Desc = "Winds back your last move.",`
`RvBoosters.cs:28 / class/static declarations`	`Color = HzUI.Hex("#3E8FB0"), Desc = "Shows a good next move.",`
`RvBoosters.cs:31 / class/static declarations`	`Color = HzUI.Hex("#8A4FB8"), Desc = "Pull any face-down card out onto the waste.",`
`RvBoosters.cs:34 / class/static declarations`	`Color = HzUI.Hex("#2E9C7E"), Desc = "Opens a spare slot to park one card for the rest of the level.",`

`RvTheme.cs:12 / class/static declarations`	`public static readonly Color VelvetTop = HzUI.Hex("#4A2650");`
`RvTheme.cs:13 / class/static declarations`	`public static readonly Color VelvetBottom = HzUI.Hex("#1F1024");`
`RvTheme.cs:14 / class/static declarations`	`public static readonly Color Plum = HzUI.Hex("#5B2D5E");`
`RvTheme.cs:15 / class/static declarations`	`public static readonly Color PlumDark = HzUI.Hex("#2E1533");`
`RvTheme.cs:16 / class/static declarations`	`public static readonly Color Ivory = HzUI.Hex("#FFF7EA");`
`RvTheme.cs:17 / class/static declarations`	`public static readonly Color CardEdge = HzUI.Hex("#E2CFA8");`
`RvTheme.cs:18 / class/static declarations`	`public static readonly Color Rose = HzUI.Hex("#D8435B");`
`RvTheme.cs:19 / class/static declarations`	`public static readonly Color Ink = HzUI.Hex("#2A2550");`
`RvTheme.cs:20 / class/static declarations`	`public static readonly Color Brass = HzUI.Hex("#E3AE55");`
`RvTheme.cs:21 / class/static declarations`	`public static readonly Color BrassDark = HzUI.Hex("#A8742A");`
`RvTheme.cs:22 / class/static declarations`	`public static readonly Color Copper = HzUI.Hex("#C8693B");`
`RvTheme.cs:23 / class/static declarations`	`public static readonly Color Mint = HzUI.Hex("#6FE3C8");`
`RvTheme.cs:24 / class/static declarations`	`public static readonly Color Sky = HzUI.Hex("#7FB6F0");`
`RvTheme.cs:25 / class/static declarations`	`public static readonly Color Text = HzUI.Hex("#3A2233");`

CHAPTER 10

Reskin recipe: visible menu and HUD text

Open `Scripts/ReverseSolGame.cs`, find the displayed literal `"Upside Solitaire"`, and replace its displayed value with `"Upside Solitaire Edition"`. Keep hierarchy names and technical IDs unchanged. In HzUI.Label/Button calls the first string may be an object name; change the displayed text argument. Read the method references below to distinguish construction from later refresh assignments.

Preserve interpolation braces, invariant number formatting, ON/OFF updates and line breaks. Update both initial and refresh paths, portrait and landscape. Exact helper names/unlocks/charges: Scripts/RvBoosters.cs; help and effect descriptions: Scripts/RvBoosters.cs. Level names/hints remain separate JSON content. If a title appears in art, Title property or another screen, update those visible uses deliberately without renaming GameId, namespace, resources or save keys.

Hard-coded game strings do not automatically use shared localization. To translate the interface, route every display assignment through your translation table. Dictionaries, word banks and theme pools are independent gameplay assets and need separate validation. Test help, settings, results, tutorial and helper introductions after a fresh launch.

Source	Exact current code
`ReverseSolGame.cs:52 / protected override void Build()`	`try { _file = ta ? JsonUtility.FromJson<RvLevelFile>(ta.text) : null; } catch (System.Exception e) { Debug.LogError("[ReverseSol] levels.json: " + e.Message); }`
`ReverseSolGame.cs:441 / void DomeTap(int col)`	`Toast("Under glass! Wind the " + RvCard.LongName(key) + " onto its dial to lift the dome.", RvTheme.Mint, 2f, 0.3f);`
`ReverseSolGame.cs:528 / void RingBell(int card)`	`Toast(_def.moves > 0 ? "Ding! Bell cards add moves." : "Ding! Bell cards add time.", RvTheme.Brass, 1.6f, 0.3f);`
`ReverseSolGame.cs:623 / void Lose(LoseReason why)`	`list.Add(($"Rewind ({have})", RvBoosters.Def(RvBooster.Rewind).Color, () => { _ended = false; UseRewindFromPopup(); }));`
`ReverseSolGame.Hud.cs:33 / void BuildHud()`	`_title = HzUI.Label("Title", _titleBox, "LEVEL 1", 64, Color.white);`
`ReverseSolGame.Hud.cs:35 / void BuildHud()`	`_subtitle = HzUI.Label("Sub", _titleBox, "", 34, RvTheme.Brass, TextAnchor.MiddleCenter, false);`
`ReverseSolGame.Hud.cs:43 / void BuildHud()`	`_goalText = HzUI.Label("Goal", _goalCard, "", 34, Color.white, TextAnchor.MiddleLeft, false);`

`ReverseSolGame.Hud.cs:53 / void BuildHud()`	`_movesText = HzUI.Label("Moves", _statsRow, "", 34, Color.white, TextAnchor.MiddleCenter, false); _movesText.fontStyle = FontStyle.Bold;`
`ReverseSolGame.Hud.cs:54 / void BuildHud()`	`_timeText = HzUI.Label("Time", _statsRow, "", 34, Color.white, TextAnchor.MiddleCenter, false); _timeText.fontStyle = FontStyle.Bold;`
`ReverseSolGame.Hud.cs:55 / void BuildHud()`	`_scoreText = HzUI.Label("Score", _statsRow, "", 34, RvTheme.Brass, TextAnchor.MiddleCenter, false); _scoreText.fontStyle = FontStyle.Bold;`
`ReverseSolGame.Hud.cs:56 / void BuildHud()`	`_streakText = HzUI.Label("Streak", _statsRow, "", 30, RvTheme.Mint);`
`ReverseSolGame.Hud.cs:67 / void BuildHud()`	`var btn = HzUI.Button("B" + i, _boosterBar, null, d.Color, Color.white, () => OnBooster(id), 40, 34);`
`ReverseSolGame.Hud.cs:70 / void BuildHud()`	`var nm = HzUI.Label("Name", btn.transform, d.Name.ToUpperInvariant(), 24, Color.white);`
`ReverseSolGame.Hud.cs:75 / void BuildHud()`	`_bBadge[i] = HzUI.Label("N", badge.transform, "0", 32, RvTheme.Brass); _bBadge[i].rectTransform.Stretch();`
`ReverseSolGame.Hud.cs:80 / void BuildHud()`	`var lt = HzUI.Label("Lv", lockP.transform, "LEVEL " + d.Unlock, 24, Color.white);`
`ReverseSolGame.Hud.cs:161 / void UpdateHud(bool full)`	`_title.text = "LEVEL " + _levelNum;`
`ReverseSolGame.Hud.cs:162 / void UpdateHud(bool full)`	`_subtitle.text = (_def.name ?? "") + (S.OpenFace ? " · Open Face" : "");`
`ReverseSolGame.Hud.cs:174 / void UpdateHud(bool full)`	`_movesText.text = movesLimit > 0 ? "Moves " + S.Moves + "/" + movesLimit : "Moves " + S.Moves;`
`ReverseSolGame.Hud.cs:188 / void UpdateClock()`	`_timeText.text = "Time " + RvLevels.Clock(left);`
`ReverseSolGame.Hud.cs:191 / void UpdateClock()`	`else { _timeText.text = "Time " + RvLevels.Clock(sec); _timeText.color = Color.white; }`
`ReverseSolGame.Hud.cs:198 / void UpdateScore()`	`_scoreText.text = IsPortrait && streak ? "Score " + _score + " x" + _streak : "Score " + _score;`

`ReverseSolGame.Hud.cs:201 / void UpdateScore()`	`if (streak) _streakText.text = "CHIME STREAK x" + _streak;`
`ReverseSolGame.Hud.cs:214 / void UpdateBoosters()`	`_bBadge[i].text = n > 0 ? n.ToString(System.Globalization.CultureInfo.InvariantCulture) : "+";`
`ReverseSolGame.Hud.cs:215 / void UpdateBoosters()`	`if (b == RvBooster.Pocket && S != null && S.CellOn) _bBadge[i].text = "ON";`
`ReverseSolGame.Hud.cs:231 / void FloatText(string text, Vector2 at, Color color, bool single = false)`	`var l = HzUI.Label("Float", _fx, text, 54, color);`
`ReverseSolGame.Hud.cs:264 / void ShowLoading(string text)`	`var l = HzUI.Label("T", p.transform, text, 44, Color.white); l.rectTransform.Stretch(10);`
`ReverseSolGame.Hud.cs:295 / void ShowDialog(string title, Color accent, string msg, params (string, Color, Action)[] buttons)`	`var t = HzUI.Label("Title", ribbon.transform, title, 70, Color.white); t.rectTransform.Stretch(6);`
`ReverseSolGame.Hud.cs:337 / void ShowWinPopup(int stars, int bonus)`	`var stats = HzUI.Label("Stats", card, (_def.name ?? "") + "\nMoves " + S.Moves + " · Time " + RvLevels.Clock((int)_time) + " · Score " + _score + (bonus > 0 ? "\n(speed bonus +" + bonus + ")" : ""), 34, RvTheme.Text, TextAnchor.MiddleCenter, false);`
`ReverseSolGame.Hud.cs:340 / void ShowWinPopup(int stars, int bonus)`	`CardButton(card, "Next Clock", RvTheme.Copper, 150, () => { close(); NextLevel(); });`
`ReverseSolGame.Hud.cs:341 / void ShowWinPopup(int stars, int bonus)`	`CardButton(card, "Replay", RvTheme.Plum, 30, () => { close(); StartLevel(_levelNum); }, 360);`
`ReverseSolGame.Hud.cs:395 / void ShowUnlock(RvBoosterDef d)`	`var card = OpenCard("New Helper!", d.Color, new Vector2(820, 820), out var close);`
`ReverseSolGame.Hud.cs:399 / void ShowUnlock(RvBoosterDef d)`	`var n = HzUI.Label("N", card, d.Name.ToUpperInvariant(), 58, d.Color);`
`ReverseSolGame.Hud.cs:401 / void ShowUnlock(RvBoosterDef d)`	`var t = HzUI.Label("T", card, d.Desc + "\nYou get " + d.FreeCharges + " free" + (d.RefillPerLevel ? " on every level." : "."), 36, RvTheme.Text, TextAnchor.MiddleCenter, false);`
`ReverseSolGame.Hud.cs:406 / void ShowUnlock(RvBoosterDef d)`	`CardButton(card, "Great!", RvTheme.Copper, 50, () =>`

`ReverseSolGame.Hud.cs:464 / void ShowCoach(string text, float seconds)`	`var l = HzUI.Label("T", p.transform, text, 36, Color.white, TextAnchor.MiddleLeft, false);`
`ReverseSolGame.Hud.cs:489 / void OnBooster(RvBooster b)`	`Toast(d.Name + " unlocks at level " + d.Unlock, Color.white, 1.4f, 0.3f);`
`ReverseSolGame.Hud.cs:504 / void OnBooster(RvBooster b)`	`Toast("+1 " + d.Name, d.Color, 1.2f, 0.3f);`
`ReverseSolGame.Hud.cs:532 / bool UndoOne(bool feedback)`	`if (feedback) { HzAudio.Play(RvSfx.Invalid, 0.4f); Toast("Nothing to rewind yet", Color.white, 1.2f, 0.3f); }`
`ReverseSolGame.Hud.cs:567 / bool UseHint()`	`if (m == null) { HzAudio.Play(RvSfx.Invalid, 0.4f); Toast("No useful move found", Color.white, 1.4f, 0.3f); return false; }`
`ReverseSolGame.Hud.cs:683 / void StartTweezers(bool free)`	`Toast("No face-down cards left to pull", Color.white, 1.4f, 0.3f);`
`ReverseSolGame.Hud.cs:709 / void TweezersPick(Place p, int pile, int index, int card)`	`Toast("Pick a face-down card", Color.white, 1.1f, 0.3f);`
`ReverseSolGame.Hud.cs:737 / bool UsePocket()`	`if (S.CellOn) { HzAudio.Play(RvSfx.Invalid, 0.4f); Toast("The pocket is already open", Color.white, 1.3f, 0.3f); return false; }`
`ReverseSolGame.Hud.cs:746 / bool UsePocket()`	`Toast("Pocket open: drag a card into it", RvTheme.Mint, 1.6f, 0.3f);`
`ReverseSolGame.Layout.cs:74 / void BuildSlots()`	`var k = HzUI.Label("K", f.transform, "K", 40, new Color(1, 0.93f, 0.8f, 0.8f));`
`ReverseSolGame.Layout.cs:86 / void BuildSlots()`	`var a = HzUI.Label("A", _colSlot[c].transform, "A", 64, new Color(1, 1, 1, 0.16f));`
`ReverseSolGame.Layout.cs:100 / void BuildSlots()`	`_foundTarget[s] = HzUI.Label("T", tb.transform, "", 32, RvTheme.PlumDark); _foundTarget[s].rectTransform.Stretch();`
`ReverseSolGame.Layout.cs:128 / void BuildDomes()`	`_domeText[c] = HzUI.Label("Key", tag.transform, "", 38, Color.white);`
`ReverseSolGame.Layout.cs:148 / void UpdateDomes(bool animate)`	`_domeText[c].text = RvCard.RankName(key) + " " + SuitShort(RvCard.Suit(key));`
`ReverseSolGame.Layout.cs:172 / void UpdateDomes(bool animate)`	`Toast("The dome lifts!", RvTheme.Mint, 1.2f, 0.3f);`

ReverseSolGame.Layout.cs:224 / void UpdateSlotsDecor()	if (goalDial) _foundTarget[s].text = "to " + RvCard.RankNameOf(_goalParam % 16);
RvBoosters.cs:24 / class/static declarations	new RvBoosterDef { Id = RvBooster.Rewind, Name = "Rewind", Art = "booster_rewind", Icon = HzIcons.Undo, Unlock = 2, FreeCharges = 3, RefillPerLevel = true,
RvBoosters.cs:25 / class/static declarations	Color = HzUI.Hex("#C8693B"), Desc = "Winds back your last move.",
RvBoosters.cs:26 / class/static declarations	Guide = "Tap REWIND to take back the move you just made." },
RvBoosters.cs:27 / class/static declarations	new RvBoosterDef { Id = RvBooster.Loupe, Name = "Loupe", Art = "booster_loupe", Icon = HzIcons.Search, Unlock = 4, FreeCharges = 3, RefillPerLevel = true,
RvBoosters.cs:28 / class/static declarations	Color = HzUI.Hex("#3E8FB0"), Desc = "Shows a good next move.",
RvBoosters.cs:29 / class/static declarations	Guide = "Tap the LOUPE and follow the glowing card." },
RvBoosters.cs:30 / class/static declarations	new RvBoosterDef { Id = RvBooster.Tweezers, Name = "Tweezers", Art = "booster_tweezers", Icon = HzIcons.Magic, Unlock = 7, FreeCharges = 2,
RvBoosters.cs:31 / class/static declarations	Color = HzUI.Hex("#8A4FB8"), Desc = "Pull any face-down card out onto the waste.",
RvBoosters.cs:32 / class/static declarations	Guide = "Tap the TWEEZERS, then tap a face-down card to pull it out." },
RvBoosters.cs:33 / class/static declarations	new RvBoosterDef { Id = RvBooster.Pocket, Name = "Pocket", Art = "booster_pocket", Icon = HzIcons.Cards, Unlock = 11, FreeCharges = 2,
RvBoosters.cs:34 / class/static declarations	Color = HzUI.Hex("#2E9C7E"), Desc = "Opens a spare slot to park one card for the rest of the level.",
RvBoosters.cs:35 / class/static declarations	Guide = "Tap the POCKET to open a spare slot next to the waste." },
RvCardView.cs:51 / class/static declarations	var l = HzUI.Label("Rank", face.transform, rn, rn.Length > 1 ? 58 : 66, ink, TextAnchor.UpperLeft);

`RvCardView.cs:59 / class/static declarations`	`var l = HzUI.Label("Rank2", face.transform, rn, 40, new Color(ink.r, ink.g, ink.b, 0.85f), TextAnchor.UpperLeft);`
`RvCardView.cs:78 / class/static declarations`	`var big = HzUI.Label("L", inner.transform, rn, 96, RvTheme.Ivory); big.rectTransform.Stretch();`

CHAPTER 11

Artwork, fonts, audio and licences

Replace game images under Resources/reversesol through Unity while retaining resource keys, .meta GUIDs, Sprite/UI import settings, alpha, pivots and slicing. Procedural graphics in the art/view sources may need separate colour changes. Replacing one PNG does not change every fallback. Keep third-party font and other licence notices with the corresponding shipped assets.

Effects are under Resources/reversesol/Sfx. Preserve each actual load key and importer settings when replacing sounds. Supplied music: Resources/reversesol/Music/workshop_loop.ogg. Preserve that load key and verify loop/compression and the independent Music toggle. Kit UI uses supplied fonts; preserve actual licence notices and verify new glyphs.

CHAPTER 12

Local saves, reset and migration

HzSave wraps PlayerPrefs and flushes local state on focus loss/quit. WebGL persistence uses browser storage tied to the hosting origin. Clearing site data or changing origin starts a new local save; this is not cloud synchronization. Inspect HzSave/PlayerPrefs calls in the actual game source before altering progression, stars, helper charges or tutorial flags.

Verify fresh launch, a completed arena, helper use, sound options, focus loss and restart. Use a dedicated test profile for resets. Keep technical save keys stable during colour/text reskins. If changing schema or keys, implement and test a migration rather than silently discarding existing users' progress.

CHAPTER 13

Advertising integration and configuration

Asset: `Assets/Szekipapa77Ads/Resources/AdsSettings.asset`. Inspect backend, useGoogleTestIds and missingSdkFallback in the publisher-owned Ads configuration. Buyer projects include adapter/integration code and exclude external provider SDK binaries. WebGL and a missing-provider fallback simulate ads; these are not live commercial impressions.

For real Android ads install the supported provider SDK from its official source into your working project, enter your own app/placement IDs and use test ads first. Verify banner/interstitial/rewarded callbacks on supported hardware. For production use your own IDs and disable simulated missing-SDK fallback. Build tooling rejects unsafe production fallback settings. No IAP, remote backend or HLK is included in this publisher edition.

CHAPTER 14

Android build and release

Install Unity 6000.0.62f1 Android modules through Hub. Select Android, run Setup and review application ID `com.szekipapa77.reversesol`, version, scene orientation and signing. The build command writes `Builds/Android/reversesol.apk`. Confirm BuildResult.Succeeded and the actual output file; intermediate report totals are not the final APK size.

Use your own keystore/signing and distribution settings. A test-signed demo is not a signed store release. A provider-free build simulates ads; a provider test-demo does not prove production ad-account configuration. Test real hardware touch, safe areas, focus/audio, save/relaunch and reward callbacks. Keep passwords, keystores and private service credentials out of buyer packages.

CHAPTER 15

WebGL build and hosting

Install matching WebGL support. Run Setup and Build WebGL; output is `Builds/WebGL/reversesol/`. Serve index.html with all loader/framework/data/WASM/StreamingAssets files over HTTP(S), not file://. Match compression, Content-Encoding and MIME types to the host. Outer page sizing and actual scene orientation are different settings.

The publisher's playable demo is hosted on HyperCodeStore. Upload the actual runtime files rather than merely changing a URL. Test a fresh anonymous load, real pointer controls, audio after interaction, saved progress/reload and phone/desktop sizing. Rebuild and rehost when code/assets change. A successful HTTP page alone is not gameplay verification.

CHAPTER 16

Troubleshooting

Compile errors: restore matching packages and the full private Kit/Ads layer; only one game Kit belongs in a project. **Pink/invisible rendering:** run Setup and inspect Graphics and every Quality URP assignment. **No input:** classic input handling must be enabled; check canvas focus and blocking modals. **Missing resources:** restore exact paths, .meta GUIDs and importer settings.

Edits do not appear: stop Play, Save in the level editor, reload the canonical bank and check current saved level. **Text reverts:** update construction and refresh assignments from the exact table. **No live ads:** provider binaries are deliberately excluded; inspect your installed provider and fallback mode. **Scene reload exception:** retain owner-lifetime and generation guards on delayed callbacks. **WebGL fails:** inspect browser Console, compression headers and missing payloads.

CHAPTER 17

Support and final release checklist

Publisher: Szekipapa77. Support: szekipapa77@gmail.com. Include game title, exact Unity version, platform, first Console error and reproduction steps. Never send passwords/tokens.

1. Clean-import the standalone project and native package.
2. Play both scenes and replace a scene near startup.
3. Review tutorial, normal completion/retry and every helper activation plus effect.
4. Validate editor changes and test reskin text/colours in both layouts.
5. Rebuild affected Android/WebGL outputs.
6. Verify complete public APK bytes, actual browser demo, documentation and video links.
7. Keep licences and exclude all internal prompts, planning/capture/build/QA tooling from buyers.